PUCRS – Escola Politécnica
Construção de Compiladores Prova **P2** 16/05/2023

Atenção! Leia toda a prova antes de iniciar. A interpretação faz parte da avaliação

Responda todas as questões na folha fornecida, esta folha não será considerada.

A prova contém 12 pontos, mas a nota máxima é 10,0

1. ***(2,0 pontos)*** Responda V/F para cada uma das questões abaixo. Explique cada uma das suas respostas.

   a) Diferentemente dos analisadores descendentes, os analisadores ascendentes não conseguem manipular recursão à direita.
   b) As fatorações necessárias para uma gramática ser do tipo LR(1) é a eliminação de conflitos de "shift-reduce" e "reduce-reduce".
   c) Se uma especificação YACC (byaccJ) possuir conflitos era não irá compilar, pois a gramática é ambígua e o analisador não irá funcionar
   d) Consideramos verificação semântica toda restrição imposta pela linguagem que não for possível codificar em uma gramática livre de contexto.

Dada a gramática abaixo, que gera a linguagem $\{a^n b^m c\}$:

L → A B
A → a A
  | ε// prod. vazia
B → b B
  | c

2. (a) Complete o conjunto de itens CLR, a partir dos itens apresentados, ***(2,0 pontos)***; (b) mostre a tabela CLR gerada a partir do estado I_0 abaixo ***(2,0 pontos)***:

I_0:

L' → • L	/ $
L → • A B	/$
A → • a A	/b,c
A → •	/b,c

   (c) Mostre o passo a passo para reconhecer, ou não, os programas ***(2,0 pontos)***:
      a. abc
      b. bc
      c. cc

3. Na linguagem C protótipos de funções são utilizados para definir o tipo de retorno e argumentos de uma função antes de sua implementação (no mesmo formato que a declaração de métodos abstratos ou métodos de interfaces em Java). São exemplos de declarações de protótipos em C:

```
int max(int a, int b);      // maior de dois valores
int min(int a, int x);      // nomes das variáveis são opcionais
void escreve();             // a lista de parâmetros é opcional
```

   a) Escreva um trecho de especificação YACC que permita declarar uma lista de protótipos em C. Assuma que os únicos tipos válidos são "int, double e void" **(2,0 pontos)**

   b) Acrescente à sua especificação ações semânticas que permitam inserir em uma tabela de símbolos as informações sobre as funções declaradas, evitando que haja re-declaração de uma função ou de nome de parâmetro se o nome for especificado (no mesmo contexto). Não é necessário escrever o código das rotinas que implementam a tabela de símbolos, apenas explique a funcionalidade dos métodos implementados **(2,0 pontos)**.

Algoritmos

Parser CLR:

Goto(E,S): Dado um conjunto de itens I e um símbolo X, o conjunto goto(I,X) é o conjunto de todos os itens $\mathbf{A \rightarrow \alpha X \bullet \beta, a}$ onde $\mathbf{A \rightarrow \alpha \bullet X \beta, a} \in I$, e $a \in T^*$.

Closure (I): A closure de I (fechamento de I) é o conjunto de itens construidos a partir do conjunto I segundo as regras abaixo:

1. $\forall$ item $\in I \Rightarrow$ item $\in$ closure(I)
2. Se $\mathbf{A \rightarrow \alpha \bullet X \beta, a} \in$ closure(I) e $\mathbf{X \rightarrow \gamma}$ é produção, então inclui $\mathbf{X \rightarrow \bullet \gamma, b}$ na closure(I), onde **b** é o First(βa). Ou seja, o novo conjunto de símbolos associado a produção é o conjunto dos primeiros símbolos após o não terminal para o qual está se fazendo a closure, (X) seguido dos símbolos associados ao item original. Com isto se $\beta \Rightarrow \varepsilon$ então **a** fará parte de **b**.

Algoritmopara criar os conjuntos de itens CLR.

Ent: Gramática G

Saida: Conjuntos de itens* CLR.

- Aumente a gramática G gerando G' pela inclusão de uma produção S' → S, onde S é o simbolo inicial de G.
- O conjunto inicial I_0 é closure({ S' → •S, $}).
- Para cada conjunto de itens I_n e cada símbolo com ponto na frente X, crie um novo cojunto I_m que será a closure(goto(I_n, X)), caso este conjunto já não exista.

Algoritmo para criar a tabela do Parser:

Ent: Gramática e conjunto de conjuntos de itens

Sai: Tabela do parser M (ações e gotos)

- Ações:
  - Para cada conjunto de itens I_n formado por um goto(I_m, X), onde X ∈ T, fazer ação[I_m, X] = S I_n (shift I_n)
  - Para item S' → S •, $ ∈ I_n, onde S' é o símbolo inicial da gramática aumentada, fazer ação[I_n, $] = ACC (Accept)
  - Para item A → α •, a ∈ I_n, fazer ação[I_n, X] = R p (reduce da produção p = A → α) para todo X ∈ a, onde isto não conflite com a regra anterior.
- Gotos:
  - Para cada conjunto de itens I_n formado por um goto(I_m, X), onde X ∈ N, fazer goto[I_m, X] = I_n .